

NMAP BASICS	3
1. History and Development of Nmap	3
2. Purpose and Functionality	3
3. Installation and Directories in Kali Linux	3
4. Basic Commands and Syntax	4
5. Zenmap: Nmap's Graphical Front-End	5
6. Advanced Features and Scripting.....	5
7. Nmap Options and Techniques.....	5
8. Directory Breakdown in Kali Linux	5
9. Combining Nmap with Other Tools.....	6
10. Real-World Usage	6
Understanding Port States in Networking	6
1. Open Ports	6
2. Closed Ports	7
3. Filtered Ports.....	7
What Makes a Port "Filtered"?.....	8
Why Do Some Firewalls Drop Instead of Rejecting?.....	8
How Nmap Detects Filtered Ports.....	8
Port States Beyond Open, Closed, and Filtered.....	9
Practical Examples.....	9
Summary of Port States	9
TCP Flags Overview	10
How TCP Flags Interact with Ports in Nmap Scans	10
1. SYN Flag	10
2. ACK Flag	10
3. RST (Reset) Flag.....	11
4. FIN (Finish) Flag.....	11
5. Xmas Scan (Combination of FIN, URG, and PSH Flags)	11
6. NULL Scan (No Flags Set)	11
Why Some Ports Give No Response and Some Send Back RST	12
Understanding the Three-Way Handshake	12
Summary of How Flags Behave with Ports	12
Nmap Scan Types Table	13

Why Are TCP and UDP So Common?.....	20
Behavior with Nmap	20
1. ICMP (Internet Control Message Protocol)	20
2. SCTP (Stream Control Transmission Protocol)	21
3. DCCP (Datagram Congestion Control Protocol)	21
4. GRE (Generic Routing Encapsulation).....	21
5. IPsec (Internet Protocol Security)	21
6. RDP (Reliable Data Protocol)	22
7. QUIC (Quick UDP Internet Connections)	22
8. EIGRP (Enhanced Interior Gateway Routing Protocol)	22
9. IGMP (Internet Group Management Protocol).....	22
NMAP Commands	23
NMAP cheatsheet.....	27

NMAP BASICS

Nmap, short for **Network Mapper**, is a popular open-source network scanning and security auditing tool. It is widely used in network discovery and vulnerability assessments. Let's dive into its history, functionalities, directories in Kali Linux, command structures, and its GUI front-ends like Zenmap.

1. History and Development of Nmap

Nmap was developed by **Gordon Lyon** (also known by his pseudonym **Fyodor**). It was first released in **nineteen ninety-seven** and has since evolved into one of the most versatile and powerful tools in network security. Nmap's development was largely driven by the need for a robust and efficient network discovery tool that could perform a variety of scans, detect open ports, and identify services running on remote machines.

The tool became popular quickly due to its capability to provide detailed information on remote systems, such as operating system detection, firewall configurations, and network topology.

2. Purpose and Functionality

Nmap serves multiple purposes, including:

- **Host discovery:** Identifying active devices in a network.
- **Port scanning:** Finding open, closed, or filtered ports on a target.
- **Service enumeration:** Detecting what services (HTTP, FTP, etc.) are running on a given port.
- **Operating system detection:** Determining the OS running on a target system.
- **Network topology mapping:** Showing the structure and layout of a network.
- **Vulnerability scanning:** Identifying potential vulnerabilities using integrated scripts.

Nmap achieves all of this by using different scanning techniques, such as TCP connect scan, SYN scan, UDP scan, and more.

3. Installation and Directories in Kali Linux

Nmap comes pre-installed in Kali Linux, as it's a fundamental tool for any penetration tester. In Kali Linux, Nmap's directory structure is located primarily under:

- **Main Directory:**

`/usr/bin/nmap` – The main executable binary for Nmap.

- **Nmap Data and Scripts:** `/usr/share/nmap` – Contains various data files and scripts.
 - **NSE (Nmap Scripting Engine) Scripts:**

`/usr/share/nmap/scripts` – This directory contains numerous NSE scripts that extend the functionality of Nmap. Scripts are categorized into:

- auth (authentication scripts)
- broadcast (network broadcast scripts)
- brute (brute-force attack scripts)
- discovery (for information gathering)
- exploit (exploit target vulnerabilities)
- external (scripts that utilize external resources)
- intrusive (aggressive scripts)
- malware (malware-related scans)
- vuln (vulnerability assessment)

4. Basic Commands and Syntax

Nmap has a wide range of commands, allowing it to perform various scanning operations. Some of the basic and commonly used commands are:

- **Scanning a Single Target:**

`nmap <target IP or hostname>`

- **Scanning Multiple Targets:**

`nmap <IP1> <IP2> <IP3>`

- **Range of IPs:**

`nmap <IP range> (e.g., 192.168.1.1-20)`

- **CIDR Notation:**

`nmap <CIDR> (e.g., 192.168.1.0/24)`

- **Port Scan:**

`nmap -p 80 <target> (to scan a specific port)`

- **Aggressive Scan:**

`nmap -A <target> (includes OS detection, version detection, script scanning, and traceroute)`

- **Operating System Detection:**

`nmap -O <target>`

- **Service Version Detection:**

`nmap -sV <target>`

- **Nmap Scripting Engine:**

```
nmap --script=<script_name> <target>
```

- **Full Scan with NSE:**

```
nmap -sC <target>
```

5. Zenmap: Nmap's Graphical Front-End

Zenmap is the **official graphical user interface (GUI) for Nmap**, making it easier to run scans and view results. It's especially useful for users who prefer a visual interface over command-line interaction.

Zenmap provides features like:

- **Scan Profiles:** You can save frequently used scan commands as profiles for quick reuse.
- **Topology Mapping:** Generates a visual network map, showing the relationship between hosts and services.
- **Detailed Results Analysis:** Zenmap allows users to drill down into scan results, making it easier to spot patterns and identify vulnerabilities.

6. Advanced Features and Scripting

Nmap includes the Nmap Scripting Engine (NSE), which is one of its most powerful features. NSE allows users to write custom scripts using Lua programming language to automate a wide variety of tasks. Some tasks that can be performed with NSE include:

- **Detecting Heartbleed vulnerability** (`ssl-heartbleed.nse`)
- **Finding open proxy servers** (`http-open-proxy.nse`)
- **Enumerating SMB shares** (`smb-enum-shares.nse`)

Scripts are divided into categories based on their purpose and level of intrusiveness.

7. Nmap Options and Techniques

- **TCP SYN Scan (-ss):** The most popular scan technique, it's fast and stealthy.
- **TCP Connect Scan (-sT):** Useful when SYN scan isn't an option.
- **UDP Scan (-sU):** Scans for open UDP ports.
- **Ping Scan (-sn):** Determines which hosts are online.
- **Stealth Scan:** Techniques like `-sN` (NULL), `-sF` (FIN), and `-sX` (Xmas) send unusual TCP flags to elicit a response.

8. Directory Breakdown in Kali Linux

- **Nmap Modules and Extensions:**

In addition to the main binaries and scripts, other directories may contain associated Nmap data files:

- **/etc/nmap**: Nmap configuration files.
- **/var/log/nmap**: Log files of scans (if configured).

9. Combining Nmap with Other Tools

Nmap can be integrated with other tools like Metasploit for automated exploitation after vulnerability detection. Tools like `ndiff` can compare the results of two scans to identify changes in the network.

10. Real-World Usage

Nmap is used in various scenarios such as:

- **Penetration Testing**: To find and exploit vulnerabilities.
- **Network Inventory**: Maintaining an updated list of active devices and services.
- **Security Auditing**: Ensuring compliance and security posture.

Understanding Port States in Networking

When a network port is scanned using tools like Nmap, it can fall into various states depending on how it responds (or doesn't respond) to the scanning attempts. Here's what the different states mean:

1. **Open Port**
2. **Closed Port**
3. **Filtered Port**
4. **Unfiltered Port**
5. **Open|Filtered Port**
6. **Closed|Filtered Port**

I'll focus on the most critical three for now — **Open**, **Closed**, and **Filtered**.

1. Open Ports

An **open port** is actively accepting TCP or UDP connections. This means there is some service running behind it that's listening for incoming requests. For example, a web server listening on **port eighty (HTTP)** or an SSH server listening on **port twenty-two**.

- **How They Behave:**

- When an Nmap scan sends a **SYN** packet to an open port, it responds with a **SYN-ACK** packet.
- If a service (like HTTP or SSH) is running, it may respond with an application-specific reply, such as sending back an HTTP header.
- **What It Means:**
 - The port is operational, and the service is running. This state is useful for attackers and administrators alike, as it indicates where services are potentially vulnerable to exploitation.
- **Why It's Open:**
 - Services on the host (like web servers, databases, FTP servers, etc.) are configured to **listen** on these ports for incoming connections.

2. Closed Ports

A **closed port** is not listening for any incoming connections. It means that no application or service is actively using that port, but it is still accessible (not filtered).

- **How They Behave:**
 - When an Nmap scan sends a **SYN** packet to a closed port, it responds with a **RST** (Reset) packet, indicating it's rejecting the connection.
 - Similarly, when a FIN, Xmas, or NULL scan is sent to a closed port, it also sends an RST.
- **What It Means:**
 - A closed port is reachable but not open for communication. This is less useful for attackers because it means there's no active service to exploit. However, administrators may want to block these ports entirely to reduce the attack surface.
- **Why It's Closed:**
 - The service that was using this port might be stopped, or the port was never used by any service in the first place. It's simply an unused port that the operating system acknowledges.

3. Filtered Ports

A **filtered port** is where things get interesting. When a port is filtered, Nmap **cannot determine** if the port is open or closed because the **packets are being dropped** by a firewall or other security device. In simple terms, the port is being **protected** by some sort of **filtering mechanism** like a firewall, IDS (Intrusion Detection System), or other security software.

- **How They Behave:**
 - When Nmap sends a **SYN**, **ACK**, or any other packet to a filtered port, **no response** is received at all.

- Sometimes, instead of the expected reply, you may receive an **ICMP packet** (like “Destination Unreachable” or “Communication Administratively Prohibited”), indicating the presence of a firewall.
- **What It Means:**
 - Filtered ports are **not accessible** directly. They are being **guarded** by a firewall that drops the packets or doesn’t allow them to reach their destination.
 - Nmap cannot definitively tell whether the port is **open** (the service is listening but the firewall is blocking visibility) or **closed** (there’s no service, and the firewall is blocking the response).
- **Why It’s Filtered:**
 - **Firewalls:** The most common reason is a **firewall** dropping or blocking packets based on security rules. Firewalls can filter packets using rules like IP address, port number, and protocol.
 - **Access Control Lists (ACLs):** Routers or security gateways may have ACLs configured to block certain traffic.
 - **Intrusion Detection/Prevention Systems (IDS/IPS):** Advanced systems may actively block Nmap scans to prevent reconnaissance.

What Makes a Port “Filtered”?

- **Packet Dropping:** A filtered port means that the firewall or filtering device is **dropping** packets silently. This is why, when Nmap sends a SYN packet to a filtered port, there is **no response** at all — no RST, no SYN-ACK, just **silence**.
- **Firewall Configurations:** A firewall rule can be configured to allow, block, or drop packets based on the **source IP address, destination port, protocol**, or other parameters. When it’s set to drop, the firewall doesn’t send any notification to the sender (Nmap), resulting in a “filtered” port state.
- **ICMP Responses:** In some cases, the firewall might send back an **ICMP Type 3 Code 13** packet (“Communication Administratively Prohibited”), which indicates that the packets are being blocked at the firewall level. Nmap will then classify the port as filtered.

Why Do Some Firewalls Drop Instead of Rejecting?

There’s a strategic reason for this: **Security through obscurity**. By dropping packets silently, the firewall avoids giving away information to attackers. If a port responded with an RST, the attacker would know the port exists and is closed. If it responded with an ICMP error, the attacker would know that a firewall is present. By not responding at all, it’s harder for the attacker to guess the network configuration.

How Nmap Detects Filtered Ports

Nmap uses multiple methods to identify filtered ports:

1. **ACK Scan (-sA):** This is primarily used to **map out firewall rules**. If a port is **filtered**, there will be no response. If it's **unfiltered**, it will respond with an RST, regardless of whether the port is open or closed.
2. **SYN Scan (-sS):** If a SYN packet sent to a port receives no response (no SYN-ACK, no RST), then the port is marked as filtered.
3. **FIN, Xmas, and NULL Scans:** Similar to SYN, if these packets receive no response from a port, the port is flagged as filtered.

Port States Beyond Open, Closed, and Filtered

- **Unfiltered:** When a port is **unfiltered**, it means that Nmap was able to send packets to the port, and it responded as expected. This doesn't mean it's open, just that there's no firewall blocking visibility.
- **Open|Filtered:** Nmap cannot determine if the port is open or filtered. This usually happens when it receives no response (like in UDP scans). It could be that the port is open, but no response is sent back, or it's filtered.
- **Closed|Filtered:** Nmap cannot determine if the port is closed or filtered. This occurs in cases where the response is ambiguous or inconsistent.

Practical Examples

1. **Firewall Rules Example:** Let's say you have a firewall rule like this:

```
iptables -A INPUT -p tcp --dport 22 -j DROP
```

1. This command tells the firewall to **drop** all incoming packets destined for port twenty-two (SSH). If Nmap scans this port, it will see **no response** and mark it as **filtered**.
2. **ICMP Blocking Example:** If you have a firewall set to block ICMP messages, Nmap will be unable to use ping or traceroute to detect live hosts, and it might classify the entire host as down or unreachable.

Summary of Port States

1. **Open Ports:** Service is active and responding.
2. **Closed Ports:** No service running, but port is reachable.
3. **Filtered Ports:** Protected by a firewall or security device, resulting in no response.

Let's delve into TCP flags and their behavior when Nmap uses them in different scans. Understanding flags like **SYN**, **ACK**, **RST**, **FIN**, and others will help you grasp how different Nmap scan types work and how they interact with open, closed, and filtered ports. I'll explain each flag and how it behaves when Nmap uses it.

TCP Flags Overview

TCP flags are special bits in the TCP header used to control the state and flow of a TCP connection. Let's break down the most commonly used flags:

1. **SYN (Synchronize)**: Initiates a connection between two hosts.
2. **ACK (Acknowledgment)**: Acknowledges the receipt of a packet.
3. **RST (Reset)**: Abruptly closes a connection.
4. **FIN (Finish)**: Gracefully ends a TCP connection.
5. **PSH (Push)**: Forces data to be sent immediately.
6. **URG (Urgent)**: Indicates that the data should be treated urgently.

Each flag serves a purpose in establishing, maintaining, or terminating a connection. However, in the context of Nmap, these flags are used creatively to manipulate how the target's ports respond, helping the scanner identify which ports are open, closed, or filtered.

How TCP Flags Interact with Ports in Nmap Scans

Let's discuss what happens when Nmap uses these flags and how ports react based on their state.

1. SYN Flag

- **Purpose**: Starts the TCP three-way handshake to establish a connection.
- **Behavior with Open Ports**: When Nmap sends a SYN packet to an open port, the port will respond with a **SYN-ACK** (synchronize-acknowledgment) packet.
- **Behavior with Closed Ports**: The closed port responds with a **RST** (reset) packet, indicating it's not listening.
- **Why It's Used**: SYN is used in Nmap's **SYN Scan (-sS)** to detect open ports without fully establishing a connection. It's often called a "half-open" scan.

2. ACK Flag

- **Purpose**: Confirms receipt of a packet or checks if the connection exists.

- **Behavior with Open Ports:** Open ports generally **ignore** ACK packets because there's no established connection yet.
- **Behavior with Closed Ports:** Closed ports typically respond with an **RST** packet.
- **Why It's Used:** The **ACK Scan (-sA)** doesn't tell you if a port is open or closed; it's used to map out firewall rules and see if a port is **filtered** (no response) or **unfiltered** (RST response).

3. **RST (Reset) Flag**

- **Purpose:** Abruptly terminates an existing connection or rejects incoming packets.
- **Behavior with Open Ports:** When an RST packet is received by an open port, the connection attempt is dropped.
- **Behavior with Closed Ports:** A closed port may also send an RST to indicate it's not open for new connections.
- **Why It's Used:** Nmap sends RST packets after SYN-ACK responses during a SYN scan to terminate the half-open connection. It's used to avoid detection by making it look like no connection was fully established.

4. **FIN (Finish) Flag**

- **Purpose:** Signals the end of a connection gracefully.
- **Behavior with Open Ports:** Open ports should **ignore** FIN packets, according to the TCP/IP RFC standard.
- **Behavior with Closed Ports:** Closed ports respond with an **RST** packet because the connection wasn't established in the first place.
- **Why It's Used:** In **FIN Scans (-sF)**, Nmap uses FIN packets to determine open ports based on the absence of responses.

5. **Xmas Scan (Combination of FIN, URG, and PSH Flags)**

- **Purpose:** Sends a packet with FIN, PSH, and URG flags set simultaneously, lighting up the packet "like a Christmas tree."
- **Behavior with Open Ports:** Open ports **ignore** these unusual packets, as they don't trigger the typical TCP state machine.
- **Behavior with Closed Ports:** Closed ports send **RST** packets, indicating the packet wasn't expected.
- **Why It's Used:** **Xmas Scans (-sX)** are another stealthy method to check for open ports without triggering typical detection mechanisms.

6. **NULL Scan (No Flags Set)**

- **Purpose:** Sends a TCP packet with no flags set.
- **Behavior with Open Ports:** Open ports **ignore** these packets.

- **Behavior with Closed Ports:** Closed ports respond with **RST**, as the packet is unexpected.
- **Why It's Used: NULL Scans (-sN)** are used to bypass basic firewall rules that filter out regular SYN packets.

Why Some Ports Give No Response and Some Send Back RST

The behavior of different ports depends on the **operating system** and **TCP/IP stack implementation**. Here's why some ports behave differently:

1. **No Response (Silent):**
 - a. This usually happens with **open ports** during FIN, Xmas, and NULL scans. The RFC-compliant behavior for open ports is to ignore such packets, which is why they seem to be "silent."
 - b. No response can also indicate that the port is **filtered** by a firewall, which is deliberately dropping packets without notifying the sender.
2. **RST Response:**
 - a. When a closed port receives a packet it doesn't expect (like SYN without a proper connection, or a random FIN/ACK packet), it sends back an **RST** packet to indicate that the connection should be terminated or the packet is invalid.
 - b. This is the default behavior for closed ports to reject connections.
3. **SYN-ACK Response:**
 - a. This response is seen only during SYN scans or TCP Connect scans. It indicates that the port is **open** and willing to establish a connection.

Understanding the Three-Way Handshake

To put everything in context, let's review the standard **three-way handshake** that happens during a normal TCP connection setup:

1. **SYN:** The client sends a SYN packet to the server to start the connection.
2. **SYN-ACK:** The server, if the port is open, replies with a SYN-ACK, acknowledging the request.
3. **ACK:** The client sends an ACK packet to confirm, establishing the connection.

If any of these steps are interrupted (e.g., if a RST is sent instead of ACK), the connection doesn't establish properly. Nmap leverages this property by sending or withholding specific flags to see how the target responds and to deduce port states.

Summary of How Flags Behave with Ports

- **Open Ports:** Tend to ignore packets with FIN, Xmas, or NULL flags.
- **Closed Ports:** Send RST responses for most unexpected packets.
- **Filtered Ports:** May drop packets altogether, resulting in no response.
- **SYN-ACK:** Only open ports send this in response to SYN packets, indicating they're listening.

Nmap Scan Types Table

		Interaction with Open Ports	Interaction with Closed Ports	Nmap Command	
Scan Type	Purpose				Additional Details
SYN Scan	Stealth scan to detect open ports without completing handshake	No response or SYN-ACK	RST	nmap -sS <target>	Default and most commonly used. Requires root privileges.
TCP Connect Scan	Full TCP handshake scan for port status	Full connection established	RST	nmap -sT <target>	Used when SYN scan isn't available. Less stealthy.
UDP Scan	Scan for open UDP services	No response or application-specific response	ICMP Port Unreachable	nmap -sU <target>	Slower than TCP scans due to nature of UDP.
FIN Scan	Stealth scan to detect open ports	No response	RST	nmap -sF <target>	Bypasses some firewalls and IDS. Ineffective on Windows.
ACK Scan	Map firewall rules and determine filtered/unfiltered ports	RST	RST	nmap -sA <target>	Does not determine open/closed ports. Checks firewall states.
Xmas Scan	Stealth scan with multiple flags (FIN, URG, PSH)	No response	RST	nmap -sX <target>	Like FIN scan, stealthy. Called "Xmas" because flags are "lit".
NULL Scan	Stealth scan with no flags	No response	RST	nmap -sN <target>	Confuses firewalls; ineffective on Windows systems.
Window Scan	Scan based on TCP window size variations	Detectable variations in window sizes	No variations in window sizes	nmap -sW <target>	Rarely used today; unreliable on modern systems.
Maimon Scan	Exploit FIN/ACK combinations to detect open ports	No response	RST	nmap -sM <target>	Uncommon; used for systems with non-standard TCP stack.

Idle Scan	Super stealthy scan using a zombie host	Zombie's IP ID changes if port is open	No change in IP ID	nmap -sI <zombie IP> <target>	Very stealthy, hides the origin of the scan.
IP Protocol Scan	Identify supported IP protocols	Response depends on specific protocols	ICMP Protocol Unreachable	nmap -sO <target>	Useful for non-TCP/UDP services.
SCTP INIT Scan	Scan for SCTP services (like SYN for SCTP)	INIT-ACK	ABORT	nmap -sY <target>	SCTP protocol is used in telecommunication systems.
SCTP COOKIE-ECHO Scan	Specialized SCTP scan for port state	COOKIE-ECHO response	ABORT	nmap -sZ <target>	Less common; used mainly in SCTP networks.
Fragmentation Scan	Split packets to evade firewalls and IDS	Variable based on target firewall/IDS rules	Variable	nmap -f <target>	Effective for bypassing basic filtering rules.
Version Detection Scan	Identify services and versions on open ports	Probes specific responses for service identification	No meaningful response	nmap -sV <target>	Essential for gathering detailed service info.
OS Detection	Detect operating system type	Analyzes TCP/IP stack fingerprint	No response if filtered	nmap -O <target>	Uses TCP/IP header behavior to guess OS.
Ping Scan	Check for live hosts without port scanning	ICMP Echo Reply or TCP ACK response	No response if host is down	nmap -sn <target>	Ideal for host discovery.
ARP Scan	Detect active hosts on local network	ARP Reply	No response	nmap -PR <target>	Used only in local networks (Ethernet).
IPv6 Scans	Scan for services using IPv6 addresses	Similar to regular scans with IPv6	Similar to regular scans with IPv6	nmap -6 <target>	Adapted for the IPv6 protocol.
Script Scan	Run NSE scripts for additional info gathering or attacks	Varies based on the script	Varies	nmap -sC <target>	Useful for vulnerability scanning.

Nmap offers a wide variety of scan types, each suited for different scenarios like stealth scanning, firewall evasion, vulnerability testing, or service detection. Here's a comprehensive list of the major Nmap scan types, their use cases, and their command structures:

1. SYN Scan (-sS)

Purpose: This is the most common and default scan. It's a half-open scan that doesn't complete the TCP handshake. It's designed to detect open ports stealthily.

How It Works: Sends a SYN packet to the target. If the port is open, the target replies with a SYN-ACK. Nmap immediately sends an RST to tear down the connection, avoiding a full handshake.

Command:

```
bash
```

```
nmap -sS <target>
```

2. TCP Connect Scan (-sT)

Purpose: Used when you don't have root privileges. It performs a full TCP connection and is less stealthy compared to SYN scan.

How It Works: Completes the three-way TCP handshake (SYN, SYN-ACK, ACK) and then closes the connection.

Command:

```
bash
```

```
nmap -sT <target>
```

3. UDP Scan (-sU)

Purpose: To find open UDP ports on a target. Useful for scanning services like DNS (port fifty-three) or SNMP (port one-six-one).

How It Works: Sends a UDP packet. If the port is open, the response varies (application-specific). If closed, the target responds with ICMP port unreachable.

Command:

```
bash
```

```
nmap -sU <target>
```

4. FIN Scan (-sF)

Purpose: A stealthy scan used to detect open ports when SYN scans are filtered.

How It Works: Sends a packet with just the FIN flag. Open ports ignore it, and closed ports respond with RST.

Command:

```
bash
```

```
nmap -sF <target>
```

5. ACK Scan (-sA)

Purpose: Used to map firewall rules and identify if ports are filtered.

How It Works: Sends an ACK packet. No response means the port is filtered, and RST response means it's unfiltered.

Command:

```
bash
```

```
nmap -sA <target>
```

6. Xmas Scan (-sX)

Purpose: A stealthy scan similar to FIN, used to identify open ports on systems that don't comply with the TCP/IP RFC.

How It Works: Sets FIN, PSH, and URG flags in the TCP header. Open ports ignore, and closed ports send back RST.

Command:

```
bash
```

```
nmap -sX <target>
```

7. NULL Scan (-sN)

Purpose: Another stealthy scan that sends packets with no flags set, used to evade firewalls and IDS.

How It Works: Open ports ignore it, and closed ports send RST.

Command:

```
bash
```

```
nmap -sN <target>
```

8. Window Scan (-sW)

Purpose: Used to differentiate between open and closed ports based on TCP window size. Not very reliable today.

How It Works: Exploits variations in TCP window size for open/closed ports.

Command:

```
bash
```

```
nmap -sW <target>
```

9. Maimon Scan (-sM)

Purpose: Similar to FIN and Xmas scans but uses a FIN/ACK combination, targeting systems that react abnormally.

How It Works: Sends a FIN/ACK packet. Response analysis depends on TCP stack implementation.

Command:

```
bash
```

```
nmap -sM <target>
```

10. Idle Scan (-sI)

Purpose: An extremely stealthy scan that hides the source of the scan by using a third-party "zombie" host.

How It Works: Sends packets that appear to be from the zombie system to determine the port status on the target.

Command:

```
bash
```

```
nmap -sI <zombie IP> <target>
```

11. IP Protocol Scan (-sO)

Purpose: Used to detect which IP protocols (ICMP, TCP, UDP, etc.) are supported on the target.

How It Works: Sends packets for each IP protocol and checks for ICMP "protocol unreachable" responses.

Command:

```
bash
```

```
nmap -sO <target>
```

12. SCTP INIT Scan (-sY)

Purpose: For scanning Stream Control Transmission Protocol (SCTP) services, a protocol similar to TCP.

How It Works: Sends an INIT packet to the target. Response analysis similar to SYN scan.

Command:

```
bash
```

```
nmap -sY <target>
```

13. SCTP COOKIE-ECHO Scan (-sZ)

Purpose: A more specialized scan for SCTP services to identify the state of the port.

How It Works: Sends a COOKIE-ECHO packet and checks for responses.

Command:

```
bash
```

```
nmap -sZ <target>
```

14. Fragmentation Scan (-f)

Purpose: Used to evade firewalls and IDS by splitting the probe packets into smaller fragments.

How It Works: Sends fragmented IP packets to bypass packet filtering devices.

Command:

```
bash
```

```
nmap -f <target>
```

15. Version Detection Scan (-sV)

Purpose: Detects the versions of services running on open ports.

How It Works: Probes each open port with service-specific queries.

Command:

```
bash
```

```
nmap -sV <target>
```

16. Operating System Detection (-O)

Purpose: Detects the operating system and hardware details of the target.
How It Works: Uses a combination of TCP/IP stack fingerprinting methods.
Command:

bash

nmap -O <target>

17. Ping Scan (-sn)

Purpose: To check which hosts are up (active) without scanning ports.
How It Works: Sends ICMP echo requests or TCP ACK packets.
Command:

bash

nmap -sn <target>

18. ARP Scan (-PR)

Purpose: To identify active devices in a local network using ARP packets.
How It Works: Sends ARP requests and looks for ARP replies.
Command:

bash

nmap -PR <target>

19. IPv6 Scans (-6)

Purpose: Use this option to perform scans on IPv6 addresses.
How It Works: Same as normal scans, but adapted for IPv6.
Command:

bash

nmap -6 <target>

20. Script Scan (-sC)

Purpose: Runs default Nmap scripts to gather more information about open ports and services.
Command:

bash

nmap -sC <target>

These are some of the most commonly used Nmap scan types. Each one has its own unique use case, depending on your goal (port discovery, service detection, or stealth scanning).

Why Are TCP and UDP So Common?

TCP and UDP are by far the most common protocols because they serve the needs of most applications:

- **TCP** offers **reliability**, **ordering**, and **error checking**, making it ideal for applications like web browsing, email, and file transfer.
- **UDP** offers **low-latency**, **connectionless** delivery, which is preferred for real-time applications like DNS queries, VoIP, and gaming.

Behavior with Nmap

When Nmap scans a network, it usually focuses on TCP and UDP for port scanning, but it can also send **ICMP packets** and use other protocols like **SCTP** to detect more specialized services or firewall rules.

While TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are the most widely used transport layer protocols, they're not the only ones out there. There are several other protocols designed to cater to specific use cases, some of which are used in specialized applications, and others are meant for specific network environments. Let's go through a few of them:

1. ICMP (Internet Control Message Protocol)

- **Purpose:** ICMP is used mainly for **network diagnostics** and **error reporting** rather than for establishing communication sessions like TCP or UDP.
- **Example:** ping and traceroute are classic tools that use ICMP to check the reachability of a host or trace the path packets take.
- **Usage in Nmap:**
 - Nmap can perform **ICMP Echo Scans** (-PE), **ICMP Timestamp Scans** (-PP), and **ICMP Address Mask Scans** (-PM) to see if a host is alive.
- **Behavior:** ICMP doesn't have ports, only **types** and **codes**. For example, an ICMP Type 8 Code 0 is an "Echo Request" (used in ping), and Type 0 Code 0 is an "Echo Reply".

2. SCTP (Stream Control Transmission Protocol)

- **Purpose:** SCTP is a **reliable, message-oriented transport layer protocol** like TCP but with additional features, primarily used in telecommunications networks (like SS7 signaling) and some modern applications.
- **Main Features:**
 - **Multi-streaming:** It allows sending multiple message streams within a single connection.
 - **Multi-homing:** A single SCTP association can span multiple IP addresses for redundancy.
- **Usage in Nmap:** Nmap supports SCTP scanning with the `-sY` (SCTP INIT scan) and `-sZ` (SCTP COOKIE-ECHO scan) options.
- **Behavior:** SCTP has its own **INIT**, **INIT-ACK**, and **COOKIE-ECHO** packets, similar to TCP's SYN and SYN-ACK, to establish connections.

3. DCCP (Datagram Congestion Control Protocol)

- **Purpose:** DCCP is designed for applications that require **timely delivery** of data but can tolerate some packet loss, like streaming media or VoIP (Voice over IP).
- **Main Features:**
 - **Congestion control:** It provides a way to manage congestion without the overhead of reliability (like TCP).
- **Behavior:** DCCP packets have **Connection Requests**, **Connection Acknowledgments**, and **Data Packets**. DCCP does not guarantee reliable delivery, but it provides congestion-controlled delivery.

4. GRE (Generic Routing Encapsulation)

- **Purpose:** GRE is a **tunneling protocol** used to encapsulate a wide variety of network layer protocols inside virtual point-to-point connections. It's often used in VPN (Virtual Private Network) setups.
- **Main Features:**
 - GRE is used to send protocols that are not typically routable over standard IP networks, like sending IPX over IP.
- **Usage:** GRE itself doesn't have ports. Instead, it wraps entire packets for secure transmission.
- **Behavior:** GRE uses Protocol Number 47 in the IP header, instead of the standard TCP or UDP.

5. IPsec (Internet Protocol Security)

- **Purpose:** IPsec is used for **securing IP communications** by authenticating and encrypting each IP packet. It's widely used in VPNs.
- **Main Features:**
 - IPsec works in two modes: **Transport Mode** (only the payload is encrypted) and **Tunnel Mode** (the entire packet is encrypted).

- It uses **AH (Authentication Header)** and **ESP (Encapsulating Security Payload)** headers.
- **Behavior:** Instead of using standard ports, IPsec identifies its traffic using **Protocol Number 50** for ESP and **51** for AH.

6. RDP (Reliable Data Protocol)

- **Purpose:** RDP is used for **reliable, sequenced delivery of data** over packet-switched networks. Not to be confused with Remote Desktop Protocol.
- **Behavior:** It operates on **port 27** and is used for some specialized applications.

7. QUIC (Quick UDP Internet Connections)

- **Purpose:** QUIC is a **multiplexed, stream-based protocol** that runs on top of UDP, developed by Google as a replacement for TCP in certain scenarios, especially for web applications.
- **Main Features:**
 - It combines the speed of UDP with the reliability of TCP.
 - Used by modern web applications for faster data transfer.
- **Behavior:** QUIC uses UDP as its transport layer but implements its own retransmission, congestion control, and connection management at a higher layer.

8. EIGRP (Enhanced Interior Gateway Routing Protocol)

- **Purpose:** EIGRP is used as a **routing protocol** rather than for general data transport. It's used to exchange routing information between routers.
- **Behavior:** It uses Protocol Number 88 instead of ports and is used mainly in Cisco-based networks.

9. IGMP (Internet Group Management Protocol)

- **Purpose:** IGMP is used to manage **multicast group memberships**. It's used in scenarios where one-to-many communication is needed, like streaming a video to multiple receivers.
- **Behavior:** Instead of using ports, IGMP uses **Types** (like ICMP). The most common types are Membership Query and Membership Report.

NMAP Commands

Nmap 7.94SVN (<https://nmap.org>)

Usage: nmap [Scan Type(s)] [Options] {target specification}

TARGET SPECIFICATION:

Can pass hostnames, IP addresses, networks, etc.

Ex: scanme.nmap.org, microsoft.com/24, 192.168.0.1; 10.0.0-255.1-254

-iL <inputfilename>: Input from list of hosts/networks

-iR <num hosts>: Choose random targets

--exclude <host1[,host2][,host3],...>: Exclude hosts/networks

--excludefile <exclude_file>: Exclude list from file

HOST DISCOVERY:

-sL: List Scan - simply list targets to scan

-sn: Ping Scan - disable port scan

-Pn: Treat all hosts as online -- skip host discovery

-PS/PA/PU/PY[portlist]: TCP SYN/ACK, UDP or SCTP discovery to given ports

-PE/PP/PM: ICMP echo, timestamp, and netmask request discovery probes

-PO[protocol list]: IP Protocol Ping

-n/-R: Never do DNS resolution/Always resolve [default: sometimes]

--dns-servers <serv1[,serv2],...>: Specify custom DNS servers

--system-dns: Use OS's DNS resolver

--traceroute: Trace hop path to each host

SCAN TECHNIQUES:

-sS/sT/sA/sW/sM: TCP SYN/Connect()/ACK/Window/Maimon scans

-sU: UDP Scan

-sN/sF/sX: TCP Null, FIN, and Xmas scans

--scanflags <flags>: Customize TCP scan flags

-sl <zombie host[:probeport]>: Idle scan

-sY/sZ: SCTP INIT/COOKIE-ECHO scans

-sO: IP protocol scan

-b <FTP relay host>: FTP bounce scan

PORT SPECIFICATION AND SCAN ORDER:

-p <port ranges>: Only scan specified ports

Ex: -p22; -p1-65535; -p U:53,111,137,T:21-25,80,139,8080,S:9

--exclude-ports <port ranges>: Exclude the specified ports from scanning

-F: Fast mode - Scan fewer ports than the default scan

-r: Scan ports sequentially - don't randomize

--top-ports <number>: Scan <number> most common ports

--port-ratio <ratio>: Scan ports more common than <ratio>

SERVICE/VERSION DETECTION:

-sV: Probe open ports to determine service/version info

--version-intensity <level>: Set from 0 (light) to 9 (try all probes)

--version-light: Limit to most likely probes (intensity 2)

--version-all: Try every single probe (intensity 9)

--version-trace: Show detailed version scan activity (for debugging)

SCRIPT SCAN:

-sC: equivalent to --script=default

--script=<Lua scripts>: <Lua scripts> is a comma separated list of
directories, script-files or script-categories

--script-args=<n1=v1,[n2=v2,...]>: provide arguments to scripts

--script-args-file=filename: provide NSE script args in a file

--script-trace: Show all data sent and received

--script-updatedb: Update the script database.

--script-help=<Lua scripts>: Show help about scripts.

<Lua scripts> is a comma-separated list of script-files or
script-categories.

OS DETECTION:

-O: Enable OS detection

--osscan-limit: Limit OS detection to promising targets

--osscan-guess: Guess OS more aggressively

TIMING AND PERFORMANCE:

Options which take <time> are in seconds, or append 'ms' (milliseconds),

's' (seconds), 'm' (minutes), or 'h' (hours) to the value (e.g. 30m).

-T<0-5>: Set timing template (higher is faster)

--min-hostgroup/max-hostgroup <size>: Parallel host scan group sizes

--min-parallelism/max-parallelism <numprobes>: Probe parallelization

--min-rtt-timeout/max-rtt-timeout/initial-rtt-timeout <time>: Specifies probe round trip time.

--max-retries <tries>: Caps number of port scan probe retransmissions.

--host-timeout <time>: Give up on target after this long

--scan-delay/--max-scan-delay <time>: Adjust delay between probes

--min-rate <number>: Send packets no slower than <number> per second

--max-rate <number>: Send packets no faster than <number> per second

FIREWALL/IDS EVASION AND SPOOFING:

-f; --mtu <val>: fragment packets (optionally w/given MTU)

-D <decoy1,decoy2[,ME],...>: Cloak a scan with decoys

-S <IP_Address>: Spoof source address

-e <iface>: Use specified interface

-g/--source-port <portnum>: Use given port number

--proxies <url1,[url2],...>: Relay connections through HTTP/SOCKS4 proxies

--data <hex string>: Append a custom payload to sent packets

--data-string <string>: Append a custom ASCII string to sent packets

--data-length <num>: Append random data to sent packets

--ip-options <options>: Send packets with specified ip options

--ttl <val>: Set IP time-to-live field

--spoof-mac <mac address/prefix/vendor name>: Spoof your MAC address

--badsum: Send packets with a bogus TCP/UDP/SCTP checksum

OUTPUT:

- oN/-oX/-oS/-oG <file>: Output scan in normal, XML, s|<rlpt klddi3, and Grepable format, respectively, to the given filename.
- oA <basename>: Output in the three major formats at once
- v: Increase verbosity level (use -vv or more for greater effect)
- d: Increase debugging level (use -dd or more for greater effect)
- reason: Display the reason a port is in a particular state
- open: Only show open (or possibly open) ports
- packet-trace: Show all packets sent and received
- iflist: Print host interfaces and routes (for debugging)
- append-output: Append to rather than clobber specified output files
- resume <filename>: Resume an aborted scan
- noninteractive: Disable runtime interactions via keyboard
- stylesheet <path/URL>: XSL stylesheet to transform XML output to HTML
- webxml: Reference stylesheet from Nmap.Org for more portable XML
- no-stylesheet: Prevent associating of XSL stylesheet w/XML output

MISC:

- 6: Enable IPv6 scanning
- A: Enable OS detection, version detection, script scanning, and traceroute
- datadir <dirname>: Specify custom Nmap data file location
- send-eth/--send-ip: Send using raw ethernet frames or IP packets
- privileged: Assume that the user is fully privileged
- unprivileged: Assume the user lacks raw socket privileges
- V: Print version number
- h: Print this help summary page.

EXAMPLES:

```
nmap -v -A scanme.nmap.org  
nmap -v -sn 192.168.0.0/16 10.0.0.0/8  
nmap -v -iR 10000 -Pn -p 80
```

SEE THE MAN PAGE (<https://nmap.org/book/man.html>) FOR MORE OPTIONS AND EXAMPLES

NMAP cheatsheet

NMAP

Misc

- 6: Enable IPv6 scanning
- A: Enable OS detection, version detection, script scanning, and traceroute
- datadir <dirname>: Specify custom Nmap data file location
- send-eth/--send-ip: Send using raw ethernet frames or IP packets
- privileged: Assume that the user is fully privileged
- unprivileged: Assume the user lacks raw socket privileges
- V: Print version number
- h: Print this help summary page.

Output

- oN/-oX/-oS/-oG <file>: Output scan in normal, XML, s<ript kddi3, and Grepable format, respectively, to the given filename.
- oA <basename>: Output in the three major formats at once
- v: Increase verbosity level (use -vv or more for greater effect)
- d: Increase debugging level (use -dd or more for greater effect)
- reason: Display the reason a port is in a particular state
- open: Only show open (or possibly open) ports
- packet-trace: Show all packets sent and received
- iflist: Print host interfaces and routes (for debugging)
- append-output: Append to rather than clobber specified output files
- resume <filename>: Resume an aborted scan
- noninteractive: Disable runtime interactions via keyboard
- stylesheet <path/URL>: XSL stylesheet to transform XML output to HTML
- webxml: Reference stylesheet from Nmap.Org for more portable XML
- no-stylesheet: Prevent associating of XSL stylesheet w/XML output

Firewall/IDS Evasion & Spoofing

- f, --mtu <val>: fragment packets (optionally w/given MTU)
- D <decoy1,decoy2[,ME],...>: Cloak a scan with decoys
- S <IP_Address>: Spoof source address
- e <iface>: Use specified interface
- g/--source-port <portnum>: Use given port number
- proxies <url1[,url2],...>: Relay connections through HTTP/SOCKS4 proxies
- data <hex string>: Append a custom payload to sent packets
- data-string <string>: Append a custom ASCII string to sent packets
- data-length <num>: Append random data to sent packets
- ip-options <options>: Send packets with specified ip options
- ttl <val>: Set IP time-to-live field
- spoof-mac <mac address/prefix/vendor name>: Spoof your MAC address
- badsum: Send packets with a bogus TCP/UDP/SCTP checksum

Timing & Performance

- T<0-5>: Set timing template (higher is faster)
- min-hostgroup/max-hostgroup <size>: Parallel host scan group sizes
- min-parallelism/max-parallelism <numprobes>: Probe parallelization
- min-rtt-timeout/max-rtt-timeout/initial-rtt-timeout <time>: Specifies probe round trip time.
- max-retries <tries>: Caps number of port scan probe retransmissions.
- host-timeout <time>: Give up on target after this long
- scan-delay/--max-scan-delay <time>: Adjust delay between probes
- min-rate <number>: Send packets no slower than <number> per second
- max-rate <number>: Send packets no faster than <number> per second

OS Detection

- O: Enable OS detection
- osscan-limit: Limit OS detection to promising targets
- osscan-guess: Guess OS more aggressively

Target Specification

- iL <inputfilename>: Input from list of hosts/networks
- iR <num hosts>: Choose random targets
- exclude <host1[,host2][,host3],...>: Exclude hosts/networks
- excludefile <exclude_file>: Exclude list from file

Host Discovery

- sL: List Scan - simply list targets to scan
- sn: Ping Scan - disable port scan
- Pn: Treat all hosts as online -- skip host discovery
- PS/PA/PU/PY: TCP SYN/ACK, UDP or SCTP discovery to given ports
- PE/PP/PM: ICMP echo, timestamp, and netmask request discovery probes
- PO(protocol list): IP Protocol Ping
- n/-R: Never do DNS resolution/Always resolve (default: sometimes)
- dns-servers <serv1[,serv2],...>: Specify custom DNS servers
- system-dns: Use OS's DNS resolver
- traceroute: Trace hop path to each host

Scan Techniques

- sS/sT/sA/sW/sM: TCP SYN/Connect()/ACK/Window/Maimon scans
- sU: UDP Scan
- sN/sF/sX: TCP Null, FIN, and Xmas scans
- scanflags <flags>: Customize TCP scan flags
- sI <zombie host[:probeport]>: Idle scan
- sY/sZ: SCTP INIT/COOKIE-ECHO scans
- sO: IP protocol scan
- b <FTP relay host>: FTP bounce scan

Port Specification & Scan Order

- p <port ranges>: Only scan specified ports
- exclude-ports <port ranges>: Exclude the specified ports from scanning
- F: Fast mode - Scan fewer ports than the default scan
- r: Scan ports sequentially - don't randomize
- top-ports <number>: Scan <number> most common ports
- port-ratio <ratio>: Scan ports more common than <ratio>

Service/Version Detection

- sV: Probe open ports to determine service/version info
- version-intensity <level>: Set from 0 (light) to 9 (try all probes)
- version-light: Limit to most likely probes (intensity 2)
- version-all: Try every single probe (intensity 9)
- version-trace: Show detailed version scan activity (for debugging)

Script Scan

- sC: equivalent to --script=default
- script= <Lua scripts>: <Lua scripts> is a comma separated list of directories, script-files or script-categories.
- script-args= <n1=v1[,n2=v2,...]>: provide arguments to scripts
- script-args-file=filename: provide NSE script args in a file
- script-trace: Show all data sent and received
- script-updatedb: Update the script database.
- script-help= <Lua scripts>: Show help about scripts. <Lua scripts> is a comma-separated list of script-files or script-categories.



@hackinarticles



<https://github.com/Ignitetechnologies>



<https://in.linkedin.com/company/hackingarticles>

